

05. GAN Fundamentals

Overview

Generative adversarial training pits a generator against a discriminator in a minimax game. You will learn the formal objective, non-saturating loss, mode collapse, and evaluation limits of GANs. The module sets up stable training practices used again in DCGAN and diffusion fine-tuning later.

Lab: Lab 05.

Contents

1	Generative Modeling Goal	2
2	Adversarial Formulation	2
3	Training Algorithm	2
4	Mode Collapse and Instability	2
5	Jensen to Shannon Divergence View	3
6	Lab and Extensions	3
7	f-Divergence Perspective	3
7.1	Label Smoothing for D	3
8	Extended Intuition	3
8.1	The Discriminator Too Strong Problem	4
9	Numerical Walkthrough	4
10	PyTorch Implementation Notes	4
11	Local GPU Constraints	5
12	Debugging Checklist	5
13	Practice Problems	5

Module track

This module starts a new track. Later modules refer back using **Module NN** labels.



Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Generative Modeling Goal

We seek a model G that samples $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ without explicit density evaluation. Generative Adversarial Networks (GANs) [2] frame this as a two-player minimax game.

2 Adversarial Formulation

Generator $G(\mathbf{z}; \theta_G)$ maps noise $\mathbf{z} \sim p_z$ to samples; discriminator $D(\mathbf{x}; \theta_D)$ outputs $\in (0, 1)$.

$$\min_G \max_D \mathcal{V}(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))] \quad (1)$$

Optimal $D^*(\mathbf{x}) = p_{\text{data}}(\mathbf{x}) / (p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x}))$.

Note. At equilibrium, $p_g = p_{\text{data}}$ and $D^* \equiv 1/2$, the discriminator cannot distinguish real from fake.

3 Training Algorithm

Alternate k discriminator steps (typically $k = 1$) with one generator step:

1. Sample minibatch $\{\mathbf{x}^{(i)}\}$ and $\{\mathbf{z}^{(i)}\}$.
2. Update D by ascending (1) w.r.t. θ_D .
3. Update G by ascending $\mathbb{E}_{\mathbf{z}} [\log D(G(\mathbf{z}))]$ (non-saturating loss).

4 Mode Collapse and Instability

Generator may map many $\mathbf{z}$ to few modes of p_{data} , fooling D but failing to cover the distribution. Gradient vanishes when D is too strong; practical fixes appear in **Module 06** (DCGAN guidelines).

$$\mathcal{L}_G^{\text{NS}} = -\mathbb{E}_{\mathbf{z}} [\log D(G(\mathbf{z}))]. \quad (2)$$

Table 1: GAN loss variants.

Variant	Generator loss
Minimax	$\log(1 - D(G(\mathbf{z})))$
Non-saturating	$-\log D(G(\mathbf{z}))$
Wasserstein	$-\mathbb{E}[D(G(\mathbf{z}))]$ (critic)

5 Jensen to Shannon Divergence View

The optimal discriminator objective relates to JS divergence between p_{data} and p_g :

$$\mathcal{V}(D^*, G) = -\log 4 + 2 \cdot \text{JS}(p_{\text{data}} \| p_g). \quad (3)$$

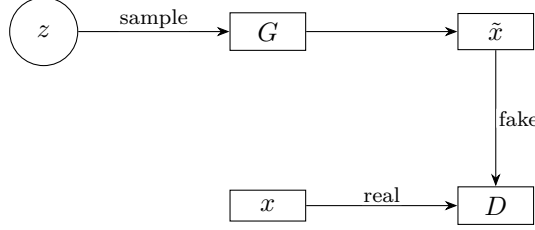


Figure 2: GAN architecture: D classifies real vs. generated.

6 Lab and Extensions

Lab 05 trains a MLP-GAN on MNIST; monitor D and G losses for collapse signs from section 4. DCGAN guidelines in **Module 06** (DCGAN guidelines) stabilize convolutional GANs. Diffusion models in **Module 07** (forward diffusion) offer an alternative generative path.

7 f-Divergence Perspective

GAN training minimizes an f -divergence between p_{data} and p_g depending on discriminator loss:

$$D_f(p_{\text{data}} \| p_g) = \mathbb{E}_{p_g} \left[f \left(\frac{p_{\text{data}}(\mathbf{x})}{p_g(\mathbf{x})} \right) \right]. \quad (4)$$

KL and reverse-KL induce different mode-covering vs. mode-seeking behavior.

7.1 Label Smoothing for D

Replace hard 0/1 targets with 0.9/0.1 to prevent overconfident discriminators.

8 Extended Intuition

GANs frame generation as a two-player game: generator G maps noise $\mathbf{z}$ to fake samples, discriminator D tries to label real vs fake. The minimax objective pushes G toward the data distribution while D sharpens its decision boundary. In practice both networks train with separate Adam optimizers and non-saturating generator loss to avoid vanishing gradients when D is too strong.

Mode collapse happens when G finds a small set of outputs that fool D repeatedly. Training is fragile: if D learns too fast, G gets no useful gradient; if G learns too fast, D sees trivial artifacts. There is no single loss number that means "good samples"; inspect outputs visually every few hundred steps.

Evaluation is harder than classification: Inception Score and FID help but require reference statistics and a pretrained classifier. For MNIST-scale labs, loss curves plus sample grids are enough to catch collapse early. DCGAN in **Module 06** adds architectural constraints that stabilize convolutional GANs on images.

8.1 The Discriminator Too Strong Problem

When D outputs $D(\mathbf{x}_{\text{real}}) \approx 1$ and $D(G(\mathbf{z})) \approx 0$ with high confidence, the generator gradient through $\log D(G(\mathbf{z}))$ vanishes (saturated sigmoid). Non-saturating loss flips the generator objective to maximize $\log D(G(\mathbf{z}))$, which still has useful gradient when D rejects fakes harshly. Heuristic: if D accuracy on a balanced real/fake batch exceeds 95% for 500+ steps, give G two updates per D step or shrink D capacity. Feature matching (optional) trains G to match intermediate D feature statistics instead of fooling the final logit; useful when samples look structured but loss oscillates. Relativistic GAN loss compares each logit to the batch mean of the opposite class; optional extension after mastering non-saturating updates here.

9 Numerical Walkthrough

MLP-GAN on MNIST flattened to 784 dims, latent $\mathbf{z} \in \mathbb{R}^{100}$, $B = 128$.

Generator: $\mathbf{z} \rightarrow \text{Linear}(100, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 512) \rightarrow \text{ReLU} \rightarrow \text{Linear}(512, 784) \rightarrow \tanh$. Discriminator: $\mathbf{x} \rightarrow 784 \rightarrow 512 \rightarrow 256 \rightarrow 1 \rightarrow \sigma$ (real probability).

Real images scaled to $[-1, 1]$ to match $\tanh$ outputs. D loss per batch:

$$\mathcal{L}_D = -\mathbb{E}[\log D(\mathbf{x})] - \mathbb{E}[\log(1 - D(G(\mathbf{z})))] \quad (5)$$

G loss (non-saturating): $\mathcal{L}_G = -\mathbb{E}[\log D(G(\mathbf{z}))]$.

Train D for 1 step, then G for 1 step, $\text{lr} = 2 \times 10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$. After 20k iterations, D accuracy on real/fake near 60 to 70% (not 100%) often indicates healthy balance. Generator params $\approx 100 \cdot 256 + 256 \cdot 512 + 512 \cdot 784 \approx 560\text{k}$.

Jensen-Shannon perspective: ideal G matches data distribution p_{data} when the game reaches equilibrium; in practice optimization finds local equilibria (mode collapse). Two time-scale update rule (TTUR): use $\text{lr} \ 2 \times 10^{-4}$ for G and 10^{-4} for D when D wins too often; not always needed on MNIST but helps on harder datasets. Track $|\mathbb{E}[D(\mathbf{x})] - \mathbb{E}[D(G(\mathbf{z}))]|$ each epoch; healthy training keeps this gap below 0.3 to 0.4 on MNIST MLP runs.

If G outputs $\tanh$ in $[-1, 1]$ but real data is $[0, 1]$, D learns intensity bias instead of structure; always match preprocessing to final activation. Latent dimension 100 on MNIST is ample; dim 10 often still works, dim 2 fails (cannot cover digit manifold). Log effective rank of $G(\mathbf{z})$ samples via PCA as a collapse diagnostic.

10 PyTorch Implementation Notes

- Separate `optim.Adam(G.parameters())` and `optim.Adam(D.parameters())`; never one optimizer on both.
- Label smoothing on real targets (0.9 instead of 1.0) reduces D overconfidence early.
- Use `detach()` on fake samples when updating D ; omit detach when updating G through D .
- `BCELoss` expects probabilities if not using `BCEWithLogitsLoss`; prefer logits version for numerical stability.
- Save fixed noise vectors `z_fixed` for comparable sample grids across epochs.

```
d_loss_real = F.binary_cross_entropy_with_logits(d_real, torch.ones_like(d_real)*0.9)
d_loss_fake = F.binary_cross_entropy_with_logits(d_fake, torch.zeros_like(d_fake))
d_loss = d_loss_real + d_loss_fake
g_loss = F.binary_cross_entropy_with_logits(d_fake, torch.ones_like(d_fake))
```

11 Local GPU Constraints

MNIST MLP-GAN at $B = 128$ uses under 500 MB VRAM; a 1650 is comfortable. If moving to 28×28 conv models previewing **Module 06**, start $B = 64$ and watch activations.

Mixed precision can destabilize GANs; keep fp32 until samples look reasonable. `DataLoader num_workers=2` on Windows often helps more than batch size tweaks for throughput.

Historical note: original GAN used simultaneous gradient descent with no label smoothing; modern recipes exist because that setup rarely works on first try. When logging samples, use `torchvision.utils.make_grid` with normalized $[0, 1]$ tensors; mis-scaled grids hide collapse until you inspect raw pixel histograms. Snapshot G every 500 steps and run D on a fixed noise batch; if $D(G(\mathbf{z}_{\text{fixed}}))$ rises monotonically while samples look worse, D is miscalibrated not G improving. Wasserstein GAN with gradient penalty (not in base lab) replaces JS divergence with Earth Mover distance; if MLP-GAN collapses repeatedly, read **Module 06** then try DCGAN before WGAN-GP. Noise injection on D input (add $\sigma = 0.01$ Gaussian to real images) slows D overfitting to memorized real samples on tiny datasets.

12 Debugging Checklist

- D loss $\rightarrow 0$, G loss flat: D too strong; train G twice per D step or weaken D architecture.
- Solid color images: G collapsed; increase latent dim, add noise, or try different seed.
- Checkerboard artifacts (conv GANs): transpose conv stride mismatches; see DCGAN guidelines in **Module 06**.
- Loss oscillates wildly: lower learning rates; ensure batch norm (if used) gets batch size > 1 .
- Generated digits all look like "1": partial collapse; inspect D gradients on fake batch.
- G loss decreases while samples worsen: D may be broken (always output 0.5); verify real/fake label assignment.

13 Practice Problems

Problem 1. Why does saturating G loss $-\log(1 - D(G(\mathbf{z})))$ vanish when $D(G(\mathbf{z})) \approx 0$?

Problem 2. Train with D steps : G steps ratio of 1:1 vs 5:1. Compare sample diversity after 10k iterations.

Problem 3. Estimate FID roughly by comparing mean/variance of flattened pixels (toy metric) between real and fake sets.

Problem 4. Log $D(\mathbf{x})$ histogram for real vs fake mid-training. What separation indicates imbalance?

Problem 5. Freeze G , train D for 1000 steps alone on balanced data. Report D accuracy; then unfreeze G and note whether G loss provides gradient.

Problem 6. Plot $D(\mathbf{x})$ vs $D(G(\mathbf{z}))$ moving averages over 5000 steps. Identify the healthy band where both stay near 0.5 to 0.8 logits.

One-sided label smoothing on fake targets (0.1 instead of 0) prevents D from becoming infinitely confident on generated samples alone. Historical minimax GAN updates both nets simultaneously; alternating D then G steps is a practical stabilizer, not a theorem requirement.

Run **Lab 05**; convolutional rules in **Module 06** and diffusion in **Module 07** offer alternative generative paths.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 05.

Topic	Primary source	Where in this PDF
Minimax GAN objective	[2]	Eq. 1
Mode collapse discussion	[2]	Section 4
Non-saturating generator loss	[2]	Section 6
Training stabilization survey	[5]	Extension section

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

- [1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *ICML*, 2017.
- [2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, et al. Generative adversarial nets. In *NeurIPS*, 2014.
- [3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [4] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *ICLR*, 2016.
- [5] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, et al. Improved techniques for training GANs. In *NeurIPS*, 2016.